

Lucrarea de laborator 1

MEDIUL DE PROGRAMARE LABVIEW

SCOPUL LUCRĂRII:

Scopul acestei lucrări este de a face o introducere în mediul de programare grafică LabVIEW. Se va prezenta conceptul de instrument virtual (VI), cum se creează un instrument virtual și se vor realiza aplicații simple de instrumentație virtuală.

NOȚIUNI TEORETICE

1. Introducere în mediul LabVIEW

Mediul LabVIEW (**L**aboratory **V**irtual **I**nstrument **E**ngineering **W**orkbench) dezvoltat de National Instruments este unul dintre primele limbaje de programare grafică ce permite realizarea de aplicații de achiziție de date, devenind un standard în programarea aplicațiilor de testare, măsurare și automatizări industriale. LabVIEW permite construirea de instrumente în interiorul computerului PC, cu performanțe și abilități ce nu pot fi atinse de instrumentele clasice. Împreună cu plăcile de intrare-ieșire (I/O), el permite generarea fluxurilor de informație digitală sau analogică pentru automatizări și crearea sau simularea aparaturii de laborator (voltmetre, osciloscop, analizoare de spectru, generatoare de semnal, etc.).

LabVIEW utilizează terminologii, idei și reprezentări familiare tehnicienilor și inginerilor, iar programarea aplicațiilor se face mai mult prin simboluri grafice decât prin instrucțiuni sub formă de text specifice programării clasice. Pentru scrierea programelor în LabVIEW se utilizează limbajul grafic de generația a 5-a, numit **G**. Programarea într-un astfel de limbaj grafic se face prin asamblarea elementelor componente prin intermediul unor fire, fiind astfel mai ușor de înțeles și depanat și de persoane cu mai puțină experiență în programare.

Codului grafic al aplicației descris în diagrama bloc în LabVIEW este executat prin conversia acestuia într-un cod mașină optimizat, prin intermediul unui compilator grafic. De fiecare dată când o aplicație creată anterior este deschisă în LabVIEW, codul sursă grafic al aplicației este disponibil, el putând fi analizat și eventual modificat foarte ușor. La fiecare comandă de rulare a programului se realizează o nouă compilare a codului.

Tehnica programării grafice, bazate pe fluxul de date, permite execuția paralelă și multitasking a diferitelor părți ale codului unui program. Limbajele de programare clasice bazate pe instrucțiuni de tip text sunt prin construcție secvențiale, instrucțiunile fiind executate în ordinea în care au fost scrise. Un alt avantaj al programării grafice în LabVIEW, bazat pe modularitatea și ierarhizarea aplicațiilor, este posibilitatea testării individuale a modulelor și reutilizarea codului.

LabVIEW rulează pe o gamă foarte largă de platforme hardware: Windows, Sun, Macintosh și SPARC stations.

1.1 Concepte de bază în LabVIEW

Orice program LabVIEW este numit „*instrument virtual*” (VI – Virtual Instrument).

Instrumentul virtual (VI) în LabVIEW este un modul de program a cărui formă grafică de prezentare (interfață) este asemănătoare cu un instrument real și realizează funcțiile acestuia.

Instrumentele virtuale sunt formate din două părți distincte, dar strâns legate între ele: **panoul frontal** și **diagrama bloc**:

- **Panoul frontal** este panoul de control al instrumentului virtual sau interfața utilizator a aplicației. Prin intermediul panoului frontal se pot stabili valori de intrare și respectiv vizualiza datele. Întrucât panoul frontal este asemănător cu cele existente la aparatele reale, intrările sunt numite „controale“, iar ieșirile „indicatoare“. Se poate folosi o mare varietate de controale și indicatoare cum ar fi întrerupătoare, comutatoare, butoane, diagrame, grafice și altele, pentru a face panoul frontal ușor de identificat și de înțeles. În figura 6.1 este prezentat un exemplu de panou frontal pentru un instrument virtual destinat măsurării temperaturii.
- **Diagrama bloc** reprezintă codul sursă al instrumentului virtual și însoțește fiecare panou frontal. Construcția diagramei bloc se face sub forma unei diagrame de flux de date, utilizând limbajul grafic de programare G. Componentele diagramei bloc, cum ar fi buclele (*Loops*), structurile *Case* și funcțiile aritmetice, controalele și indicatoarele, reprezintă nodurile programului. Aceste componente sunt conectate între ele prin fire în diagrama bloc pentru a arăta traseul fluxului de date. Un bloc de instrucțiuni se execută doar când la toate intrările sunt disponibile date; rezultatele sunt furnizate în exteriorul blocului doar după execuția tuturor instrucțiunilor din cadrul acestuia. Diagrama bloc pentru un instrument virtual destinat măsurării temperaturii este prezentată în figura 1.2.

Avantajul programului LabVIEW constă în structura ierarhică a VI-urilor. Odată creat un instrument virtual, va putea fi folosit ca subinstrument virtual (*subVI*) în diagrama bloc a unui VI de nivel mai înalt. Nu există limite ale numărului de nivele dintr-o astfel de ierarhie arborescentă. De aceea, atunci când se proiectează și implementează un VI trebuie să se țină cont de natura modulară a acestuia: să poată fi utilizat atât ca program principal cât și ca subrutină în componența altui VI.

Denumirea subVI-urilor apelate de aplicația pentru măsurarea temperaturii prezentată anterior, precum și structura ierarhică a acestora (figura 1.3) se pot vedea accesând meniul **Browse** ⇒ **Show VI Hierarchy**.

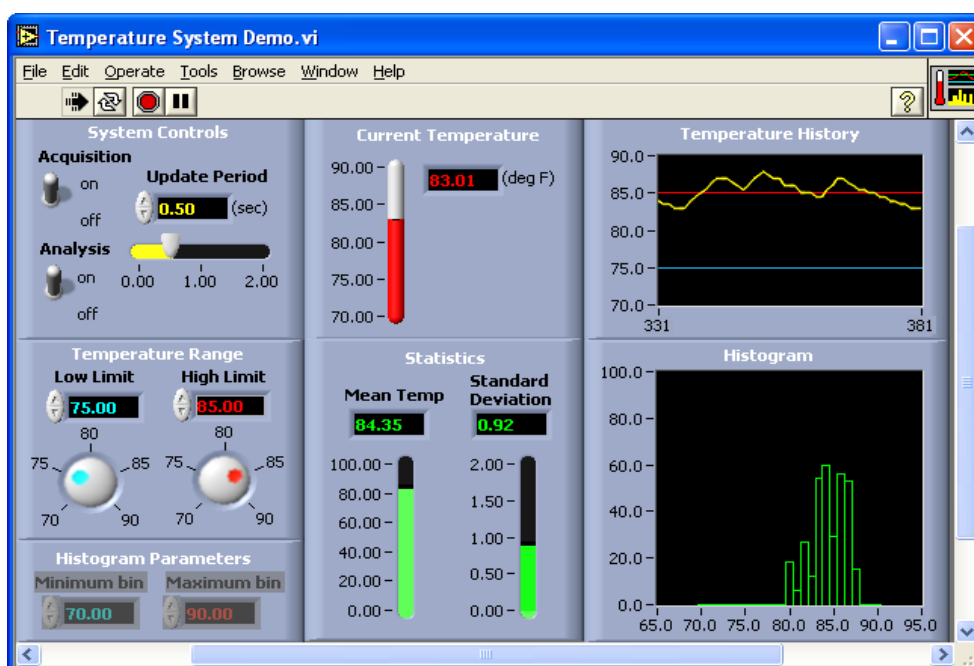


Figura 1.1 Panou frontal al unui IV pentru măsurarea temperaturii

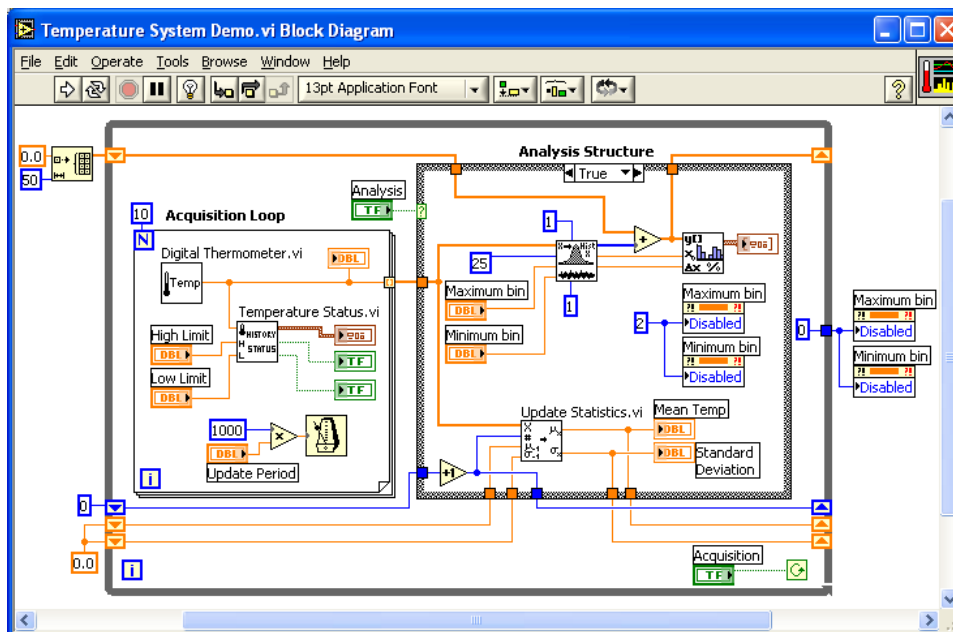


Figura 1.2 Diagrama bloc a instrumentului virtual pentru măsurarea temperaturii.

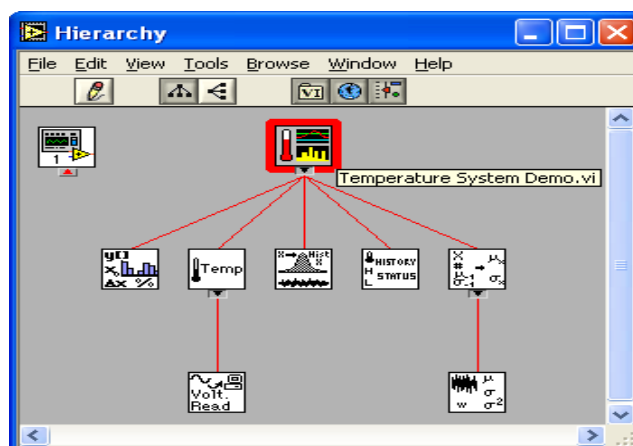


Figura 1.3 Structura ierarhică a instrumentului virtual pentru măsurarea temperaturii.

Conectarea unui VI în diagrama bloc a unui alt instrument virtual de nivel mai înalt se face printr-o *pictogramă/conector*. Terminalele conectoare indică unde trebuie legate firele pentru intrările și ieșirile instrumentului reprezentat prin pictograma respectivă.

1.2 Ferestre de lucru în LabVIEW

Programarea în LabVIEW se face în două ferestre diferite: interfața utilizator sau *panoul frontal* și *diagrama* sau codul sură. Cele două ferestre se activează simultan la lansarea programului LabVIEW, una în spatele celeilalte. Trecerea de la o fereastră la alta se face prin executarea unui click cu mouse-ul pe fereastra respectivă sau cu comanda *show diagram /show panel* din meniul **Window**.

Pentru realizarea aplicației dispunem de o multitudine de unelte, funcții de bibliotecă și subinstrumente virtuale. Imediat sub meniu avem **bara de unelte** (*Tool Bar*) care conține cele mai utilizate butoane. Celelalte unelte necesare în faza de dezvoltare a aplicației se accesează din meniul **Window** și sunt: **paleta de scule** (*Tools palette*), **paleta de control** (*Controls*) și **paleta de funcții** (*Functions*). Aceste ultime două palete se activează din ferestrele Panel, respectiv Diagram, sau prin click cu tasta dreaptă a mouse-ului în interiorul ferestrei respective. În interiorul acestor palete VI-urile și funcțiile sunt grupate pe categorii funcționale.

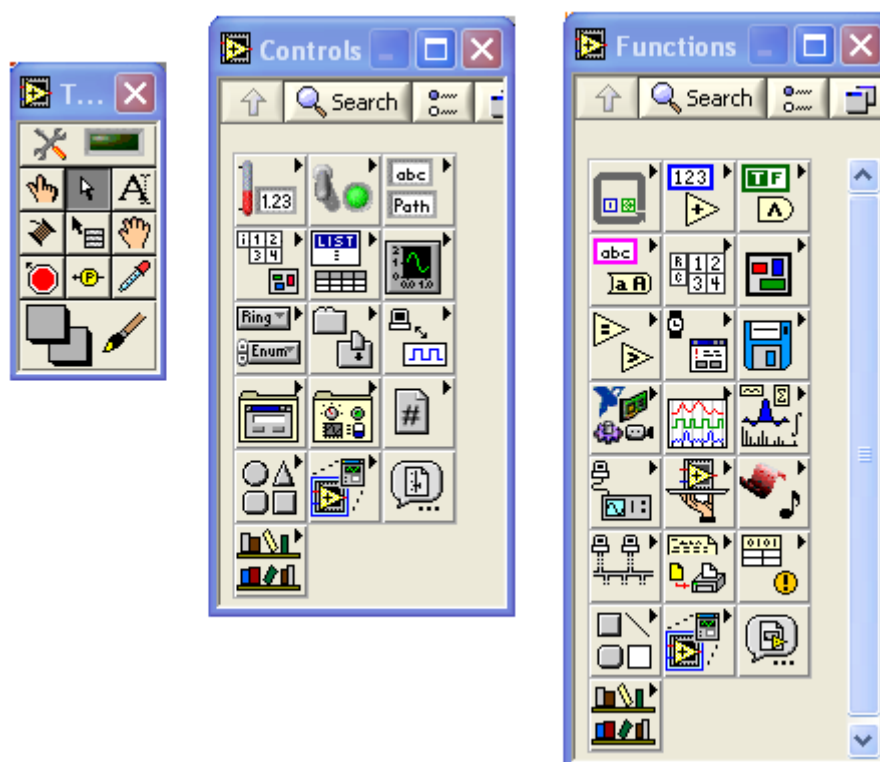


Figura 1.4 Paletele cu unelte din LabVIEW

Tabelul 1.1 Semnificația instrumentelor din meniul Tools Palette

	Selectarea automată a instrumentului – Dacă este activat, la trecerea cu mouse-ul peste un obiect din ferestrele panel și diagram se selectează automat instrumentul corespunzător din paleta de instrumente.
	Operare – schimbă valoarea unui control sau selectează textul într-un control
	Poziționare – selectează , poziționează și redimensionează obiecte
	Inscripționare – editează texte și creează etichete independente
	Cablare – leagă obiectele între ele în cadrul editării diagramei bloc a programului prin fire.
	Apelare rapidă meniul obiect – deschide meniul tip pop-up al unui obiect
	Derulare fereastră - deplasare fereastră
	Înterupere – setează puncte de întrerupere în VI-uri, funcții, etc.
	Testare (probe) – testează valorile intermediare pe conexiuni
	Copiază culoare – copiază culoarea curentă
	Setează culoarea pentru obiecte și fundal (background)

1.3. Instrucțiuni de control și tipuri de date

1.3.1 Structuri

Fluxul de date dintr-un instrument virtual (VI) este controlat cu ajutorul **structurilor**. LabVIEW dispune de opt tipuri de structuri de control al programului: structura repetitivă *While* (*While Loop*), structura repetitivă *For* (*For Loop*), structura pentru selecție multiplă *Case*, structurile secvențiale *Stacked Sequence* și *Flat Sequence* și structura eveniment *Event*, structura *Bucloa temporizată* (*Timed Loop*), *Formula Node*, și *MathScript Node*. Plasarea unei structuri în diagrama bloc se face prin selectarea sa din setul **Structs&Constants** din meniul **Functions** și încadrarea zonei care va fi repetată. Toate instrucțiunile care se repetă vor fi plasate în interiorul structurii.

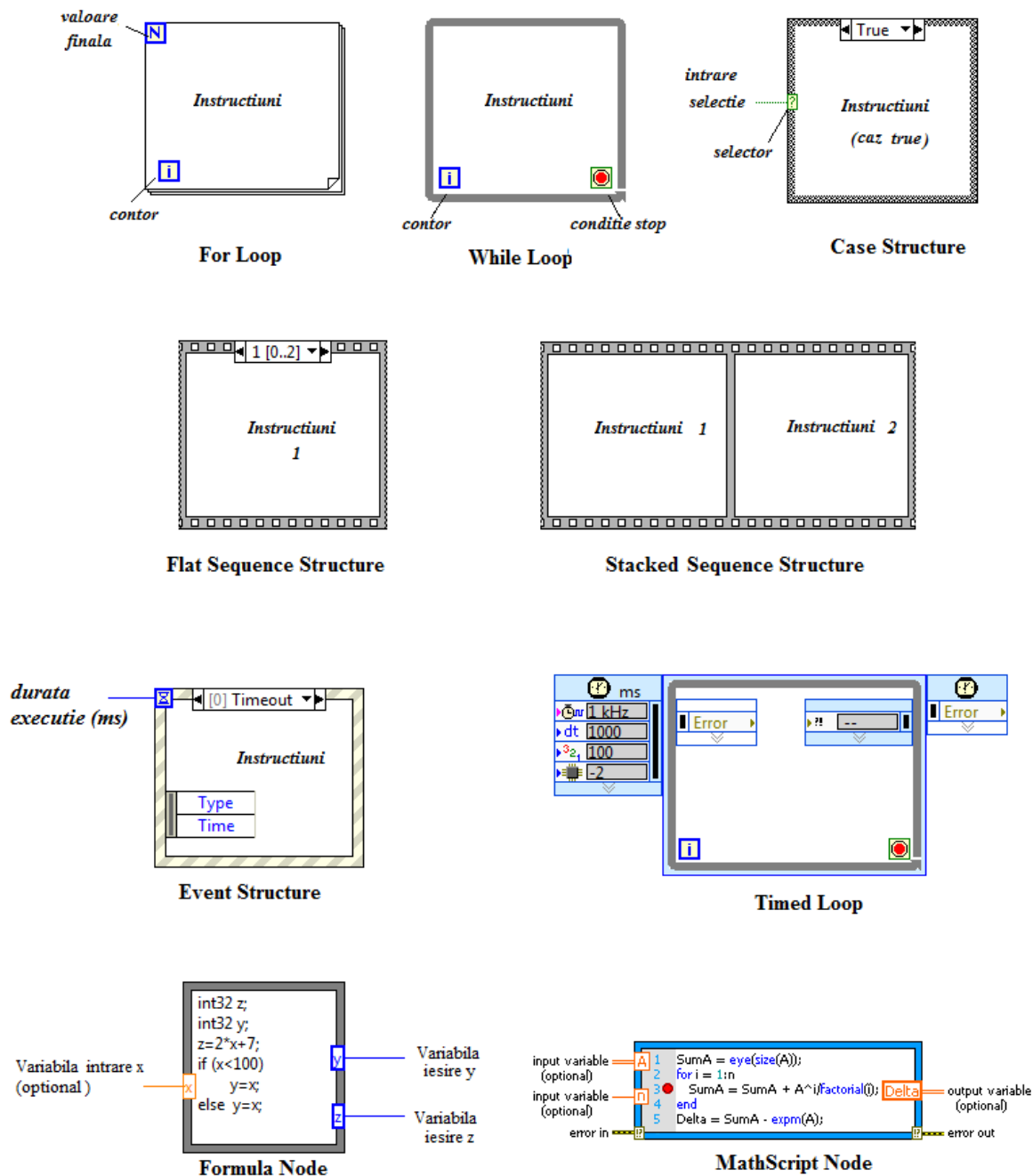


Figura 1.5. Structuri de programare în LabVIEW 8.6.

- **Structura de control Sequence (secvență)** permite construirea unui ansamblu de cadre (frame) numerotate, care sunt executate secvențial, în ordinea crescătoare a numerelor. Ea

este folosită atunci când ordinea de execuție a anumitor instrucțiuni trebuie precizată clar. Există două structuri de tip secvență: *stacked sequence* – secvență ale cărei cadre sunt suprapuse și *flat sequence* – secvență cu cadrele reprezentate unul lângă altul. În cazul secvenței cu cadrele suprapuse, datele sunt transmise de la un cadru la altul prin utilizarea unor regiștri numiți *sequence local*.

- **Structura Case** – structură de selecție cu alternative multiple, echivalenta instrucțiunilor decizionale *if* și *case* din limbajul C, – permite și ea construirea unui ansamblu de cadre numerotate, fiecare cadru corespunzând unui caz. În funcție de valoarea legată la terminalul selector, structura va executa un singur caz. Valoarea selectoare poate fi numerică sau booleană. Dacă este booleană, structura are un caz adevărat (True) și unul fals (False). Dacă este numerică, structura poate avea până la $2^{31}-1$ cazuri.
- **Ciclul For** este o structură repetitivă care execută diagrama din interiorul marginilor sale de un număr prestabilit de ori. Ciclul For dispune de două terminale: terminalul număr de execuții <<N>> (de intrare) și terminalul contor al iterațiilor <<i>> (de ieșire). Terminalul număr de execuții specifică numărul de iterații pe care trebuie să le efectueze ciclul. Terminalul contor al iterațiilor specifică numărul de iterații care au fost executate până în momentul de față. Ciclurile For și While permit folosirea regiștrilor de schimb “*Shift Register*”, care reprezintă variabile locale care transferă valorile de la o iterație la următoarea. Crearea unui registru de schimb se face executând clic dreapta cu mouse-ul pe marginea din dreapta sau stânga a ciclului și selectând **Add Shift Register** din meniul cu comenzi rapide care apare. Registrul de schimb conține o pereche de terminale așezate în opoziție pe marginile din stânga și dreapta ale ciclului.
- **Ciclul While** este o structură de repetiție clasică folosită pentru a executa diagrama din interiorul său până când valoarea booleană trimisă *terminalului condițional* (un terminal de intrare) este identică cu condiția de oprire. Această condiție poate fi setată: *stop if true* sau *continue if true*. În cazul structurii While, numărul de repetări nu este cunoscut în momentul construcției VI-ului și poate să varieze, pentru același VI, de la o rulare la alta. VI-ul verifică terminalul condițional la *sfârșitul* fiecărei iterații astfel încât **ciclul While va fi executat întotdeauna cel puțin o dată**. *Terminalul iterație* <<i>> este un terminal numeric de ieșire care conține numărul de iterații executate de ciclu până în momentul respectiv.
- Structura **Event (eveniment)** este o structură care poate avea una sau mai multe subdiagrame, corespunzătoare cazurilor care trebuie executate la apariția unui eveniment în panoul frontal al aplicației. Intrarea *time out* specifică timpul în ms pe cât structura trebuie să aștepte apariția evenimentului. Valoarea implicită este -1 ceea ce înseamnă că așteaptă la infinit (fără time out).
- Structura **Timed Loop (Bucla temporizată)** – executa instrucțiunile pe care le conține în timpul specificat. Se utilizează în aplicații în care controlul timpului de execuție al instrucțiunilor este foarte important.
- Structura **Formula Node** – executa instrucțiuni matematice și expresii definite în format similar limbajului C.
- Structura **MathScript Node** - executa scripturi scrise cu sintaxa similară limbajului Matlab

Regiștrii de schimb (disponibili pentru ciclurile While și For) sunt variabile locale care transferă valorile de la o iterație la începutul următoarei. Ei se plasează pe conturul structurii astfel: *intrarea* în stânga (o săgeată descendentă) și *ieșirea* în dreapta (o săgeată ascendentă). Pentru o execuție corectă, punctele de intrare trebuie inițializate. Regiștrii de schimb se pot configura astfel

încât să permită accesul la valorile din iterațiile precedente. Pentru fiecare iterație ale cărei valori se dorește a se revedea, trebuie adăugat un nou element la terminalul din stânga al registrului de schimb.

Diagramele - sunt *indicatoare numerice* speciale, de tip grafic, utilizate pentru afișarea unuia sau mai multor puncte. Cu ajutorul acestor indicatoare se pot reprezenta evoluția formelor de undă în timp (*waveform chart*), grafice de tip XY, sau imagini 3D. Ele pot fi selectate prin intermediul opțiunii **Graph** din meniul **Controls**. Diagramele pot afișa una sau mai multe curbe simultan. Modul de desenare a graficelor este destul de variat, utilizatorul putând să aleagă forma și culoarea de reprezentare a punctelor, modul de interpolare, etc. Diagramele permit trei moduri de actualizare a imaginii: prin derulare (*strip chart*), prin redesenare (*scope chart*) și prin baleiere (*sweep chart*). Selectarea modului dorit se poate face prin execuția unui clic dreapta cu mouse-ul pe diagramă, selectând apoi una dintre opțiunile meniului **Data Operations** -> **Update Mode**. (În modul rulare se selectează **Update Mode** din meniul cu comenzi rapid al diagramei).

1.3.2. Tipuri de date

LabVIEW permite operarea cu diverse tipuri de date (întreg, întreg fără semn, real, complex, caracter, boolean, etc.). Reprezentarea acestora în cadrul codului grafic se face diferențiat, prin nume și culori diferite, pentru o mai ușoară înțelegere a programului. Tipurile de date sunt prezentate în figura 1.6.

Transferul datelor între nodurile diagramei bloc sau între panoul frontal și diagrama bloc se realizează prin intermediul terminalelor. Acestea sunt reprezentate printr-o pictogramă prin care se specifică:

- *tipul componentei* (un control are chenarul pictogramei mai gros decât al unui indicator);
- *tipul datelor*, reprezentat prin culori diferite: întreg – albastru; real, complex – portocaliu; boolean, cale de fișier – verde; șir de caractere – roz; grupare de date cu elemente tip numeric – maro;
- *reprezentarea/precizia datelor*, realizată prin caractere alfanumerice afișate în interiorul pictogramei;
- *structură compusă tip tablou*; pictograma tabloului afișează între paranteze pătrate tipul elementelor.

Tipuri de date	Reprezentare date în meniul contextual	Reprezentare date în diagrama bloc
Întreg cu semn	 byte	
	 word	
	 long	
Întreg fără semn	 unsigned byte	
	 unsigned word	
	 unsigned long	
Real	 single precision	
	 double precision	

		extended precision	
Complex		complex single	
		complex double	
		complex extended	
Boolean			

Figura 1.6. Tipuri de date și reprezentarea acestora

Datele pot fi vehiculate între funcțiile și subinstrumentele (subVI) care intră în componența unui instrument virtual în mai multe forme:

- *scalar*,
- *array* vector unidimensional (1D array),
- matrice sau vector multidimensional (2D array),
- *cluster* - structura de date care poate să conțină variabile multiple de același tip sau diferite;
- *waveform* – structura de date specifică utilizată în aplicații de achiziție și procesare de semnale, care conține valorile măsurate, data și ora la care au fost achiziționate, timpul inițial t_0 de începere a achiziției, perioada dintre două eșantioane dt (eșantionare uniformă), informații despre semnale.

Firele de legătură între componente sunt specifice tipului de date vehiculate, și diferă prin culoare și grosime (figura 1.7), pentru o mai ușoară înțelegere a funcționării aplicației software:

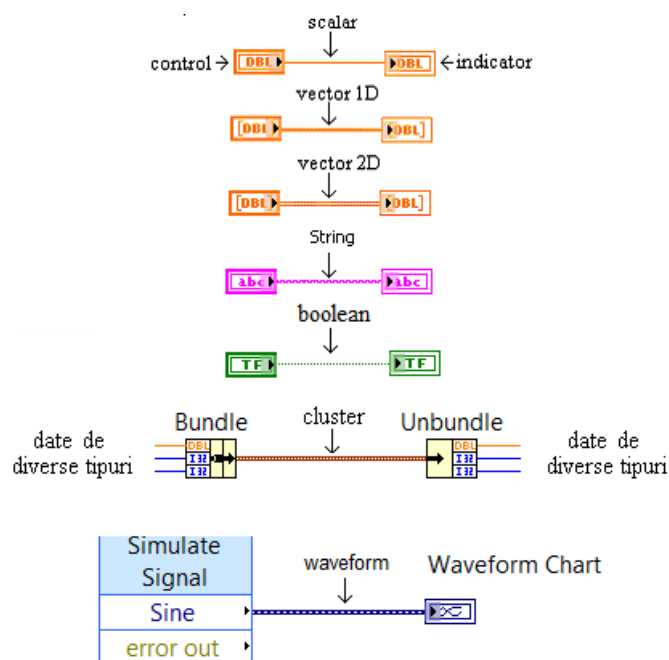


Figura 1.7. Exemple de reprezentare a firelor

LabVIEW permite gruparea mai multor date de același tip sau diferite într-o structură de tip *cluster*, folosind funcția *Bundle*. Operația inversă, de extragere a datelor dintr-un cluster, se

numește *Unbundle*. Aceste funcții se găsesc în librăria *Cluster*. Un exemplu de utilizare a datelor de tip cluster este pentru reprezentarea pe același grafic a mai multor variabile indexate.

Funcții de reprezentare grafică

Orice instrument virtual, asemenea instrumentelor clasice, necesită o componentă de afișare grafică în diverse formate – afișare valori numerice în casete, tabele sau liste; afișare mesaje text; afișare grafice diverse (grafice evoluție mărimi, grafice de tip intensitate, grafice de tip 3D, etc.); afișare indicatoare analogice, indicatoare luminoase.

De asemenea, există diverse tipuri de controale de tip numeric, boolean, text, etc. pentru controlul IV, cu grafică asemănătoare cu butoanele aparatelor clasice.

Paleta de indicatoare și controale standard în mediul LabVIEW este prezentată în figurile 1.8 și 1.9.

Indicatoare si controale grafice:

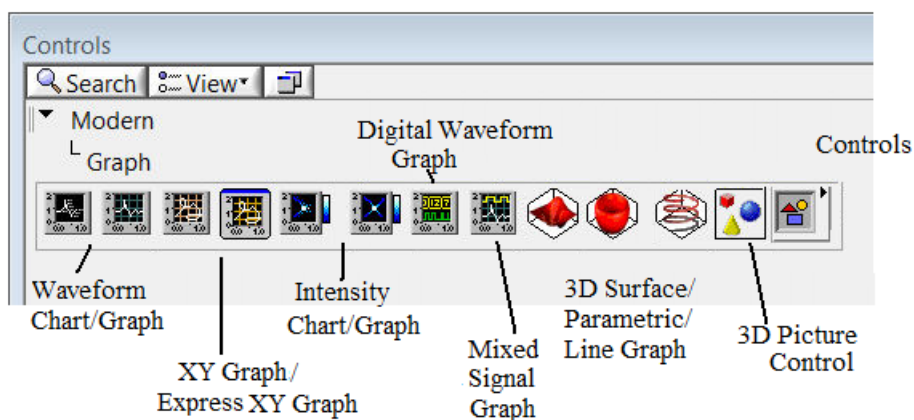
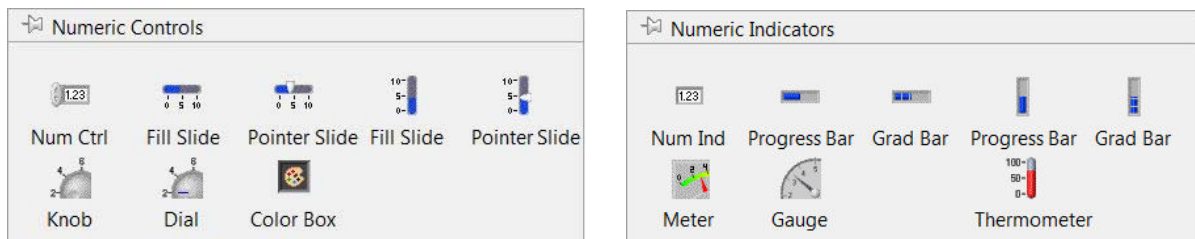


Figura 1.8. Funcții de reprezentare grafică

Controale și indicatoare numerice:



Controale și indicatoare booleene:

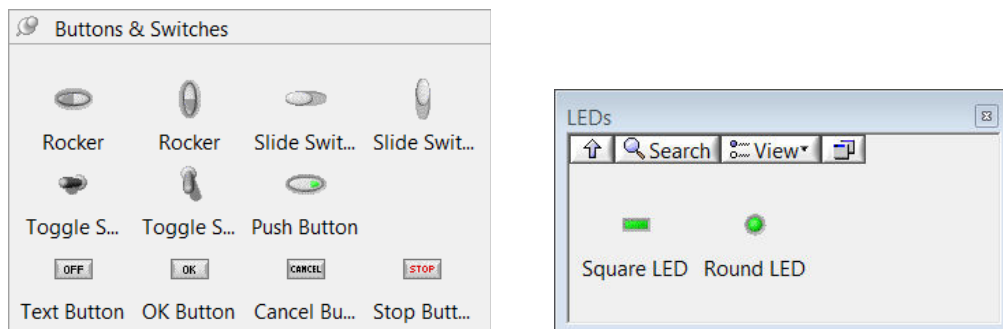


Figura 1.9. Principalele funcții de introducere și afișare date în interfața cu utilizatorul

Alte facilități de afișare și control (Tabele și liste, Casete text și Cale fișier pe harddisk, butoane tip selecție din listă, afișare/introducere vectori și matrici, etc.) sunt prezentate în figura 1.10.

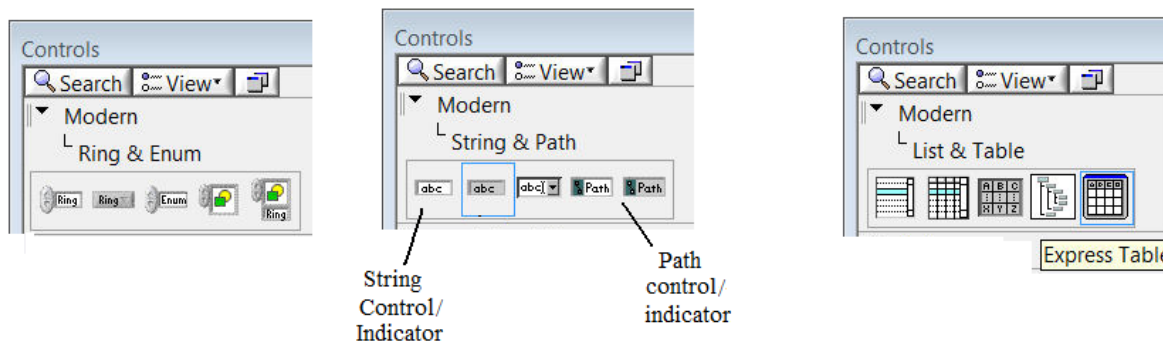


Figura 1.10. Alte facilități de introducere și afișare date în interfața cu utilizatorul.

1.4. Analiza sistemelor și a semnalelor

LabVIEW este un mediu deosebit pentru analiza semnalelor și a sistemelor. El permite dezvoltarea de programe pentru rezolvarea sistemelor de ecuații algebrice, fitarea curbelor, integrarea ecuațiilor diferențiale ordinare, calcularea derivatelor și a integralelor diverselor funcții, generare și analiză de semnal, calcularea transformatei Fourier discretă și filtrarea semnalelor. Pentru aceasta, LabVIEW conține un număr mare de librării pentru rezolvarea ecuațiilor liniare algebrice. Funcțiile pentru lucrul cu vectori și matrici se găsesc în submeniul *Array* din paleta de funcții și în submeniul *Mathematics*.

Scrierea/salvarea în fișiere și operații cu șiruri de caractere

LabVIEW oferă posibilitatea lucrului cu șiruri de caractere și operații I/O cu fișiere, necesare pentru stocarea/citirea datelor. Funcțiile și VI-urile pentru aceste operații sunt disponibile în librăriile *String* și *File I/O* din paleta de funcții (figura 1.11).

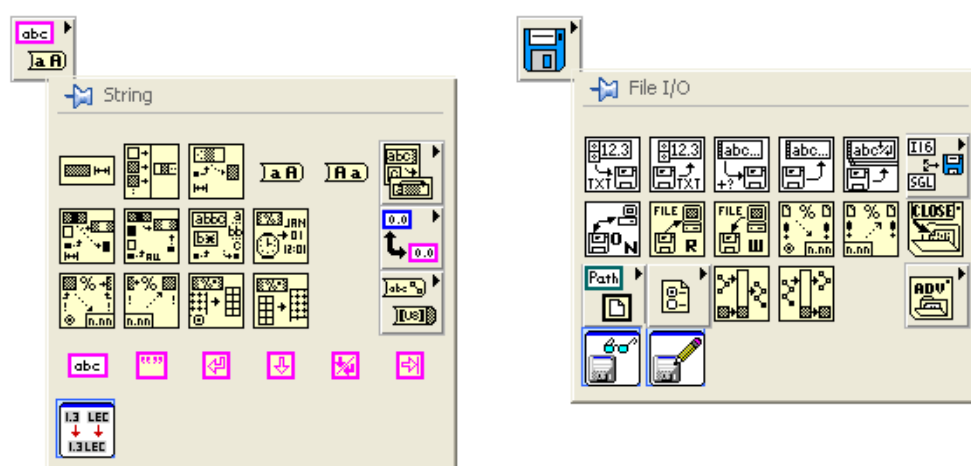


Figura 1.11 Funcțiile pentru lucrul cu șiruri de caractere și cu fișiere.

Pentru scriere/citirea datelor pe harddisk se pot folosi funcțiile simple de lucru cu fișiere sau VI-uri care pot scrie/citi toate componentele unui vector sau șir de caractere.

Scrierea datelor dintr-un vector/matrice într-un fișier text pe harddisk, pe linie sau coloană, se poate face folosind VI-ul *Write to Spreadsheet File.vi* din librăria File I/O. Datele de intrare de tip numere reale single-precision dintr-un vector (unidimensional sau multidimensional) se aplică pe intrările 1D sau 2D, sunt convertite într-un șir de caractere cu formatul specificat (formatul default este `%.3f` - numere reale cu 3 cifre după virgulă), iar apoi sunt salvate în fișierul specificat pe intrarea *file path*. Dacă nu se specifică nici un fișier, la rulare se va deschide o fereastră de tip *Open File* pentru a alege fișierul dorit. Fișierul poate fi deschis ca nou sau pentru adăugare după ultima înregistrare, opțiune accesibilă pe intrarea *append to file?*. Implicit, fișierul se va deschide ca nou. Datele pot fi scrise în fișier pe linie (*transpose?* - false) sau pe coloane (*transpose?* - true), fiind delimitate de caracterul specificat pe intrarea *delimiter* (implicit, caracterul delimitator este Tab). După salvarea datelor, fișierul este închis.

Operația inversă de extragere într-un vector/matrice a datelor dintr-un fișier se face cu VI-ul *Read From Spreadsheet File.vi*

Observație: intrările care au specificate valori implicite pot fi lăsate neconectate; în acest caz valoarea lor va fi cea implicită (*default*).

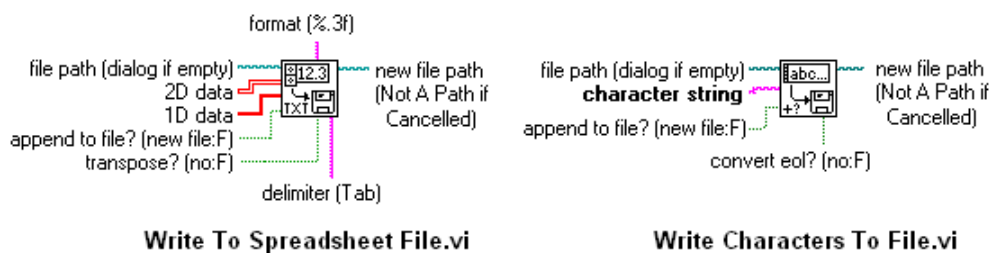


Figura 1.12 VI-uri pentru salvarea vectorilor și a șirurilor de caractere în fișiere, cu reprezentarea și semnificația conectorilor.

Pentru salvarea șirurilor de caractere într-un fișier se poate folosi VI-ul *Write Characters To File.vi*, care operează în mod asemănător.

Pentru lucrul cu șiruri de caractere (string) există diverse facilități, disponibile în librăria *String*, cum ar fi funcții pentru căutare într-un șir, concatenare șiruri, conversie din string în alte formate și invers, etc.

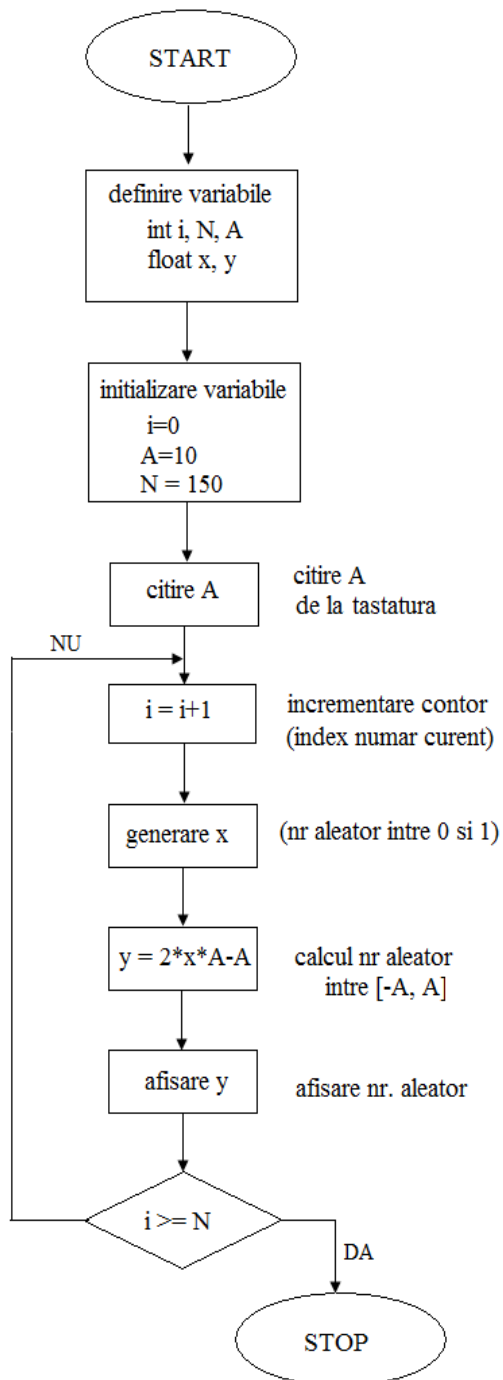
Un alt capitol la care LabVIEW oferă un număr mare de funcții și VI-uri dedicate este comunicația pentru controlul instrumentelor și comunicația între aplicații. Fiind orientat în special pentru realizarea de aplicații de măsurare și automatizare, în bibliotecile sale există funcții și VI-uri pentru comunicația cu echipamentele hardware pe principalele tipuri de interfețe: VISA, VXI/PXI, RS-232, RS-485, GPIB (IEEE 488), PCI, USB, Fireware, Ethernet, Driver de instrument.

Funcțiile pentru controlul instrumentelor și pentru comunicație sunt grupate în librăriile *Communication* și *Instrument I/O*. Pentru plăcile de achiziție de date interne produse de National Instruments există drivere care se instalează separat (NiDAQ drives). Când sunt instalate și driverele, funcțiile pentru controlul plăcilor de achiziție sunt grupate în librăria *NI Measurement*.

LabVIEW oferă de asemenea facilități pentru realizarea de aplicații distribuite, la care comunicația se poate realiza prin rețeaua de calculatoare folosind protocol TCP/IP, sau prin internet. În acest sens există disponibile în librăriile sale mecanisme și funcții pentru comunicația pe rețea (TCP/IP, UDP), tehnologii precum .NET, Active X, SMTP Email, VI Server sau DataSocket Technology.

Exemplu implementare algoritm in C si LabVIEW pentru generarea a 150 numere aleatoare in intervalul $[-A, A]$.

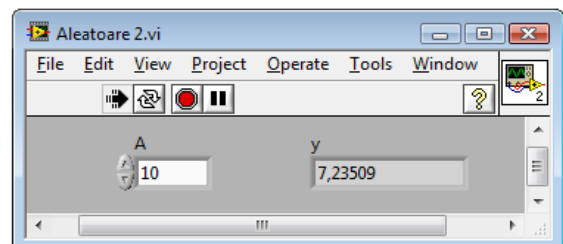
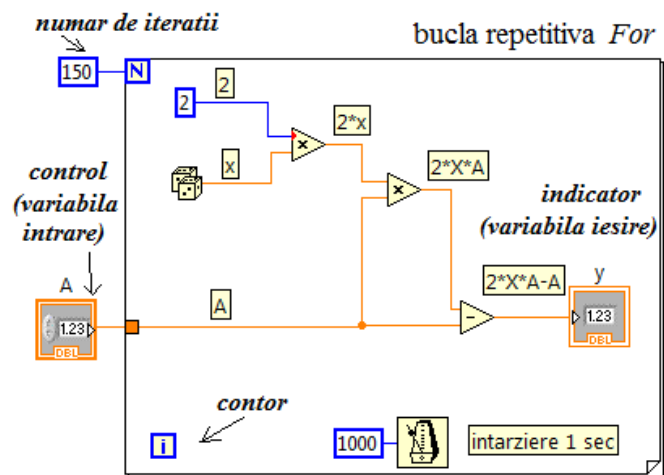
ALGORITM:



```

//Program C
//Definire variabile
Double x, y;
Int i, N;
Int A=10;
i=0;
N=150;
Void main ()
{
    clrscr();
    A=getch ();
    for i=0 to N
    {
        x=random (0,1);
        y= 2 *x*A-A;
        printf(„Numar aleator = %lf.3”, y);
        // generare 1 nr. aleator pe secunda
        delay(1000); // intarziere 1000ms
    }
}
  
```

Program LabVIEW:



1.5. DESFĂȘURAREA LUCRĂRII - APLICAȚII

A. Generarea unor programe (VI) simple -

1. Construiți un VI care să genereze numere aleatoare a căror valoare variază într-un interval $[-A, +A]$, unde valoarea A poate fi reglată de utilizator, și să le afișeze într-o diagramă cu actualizare de tip redesenare. Pentru generarea numerelor aleatoare se va folosi generatorul de numere aleatoare cuprinse între 0 și 1 din LabVIEW. Dacă notăm cu x aceste numere aleatoare și cu y numerele din intervalul $[-A, +A]$, atunci numerele y se pot calcula cu relația:

$$y = 2Ax - A$$

Panoul și schema bloc ale aplicației sunt prezentate în figura 1.13

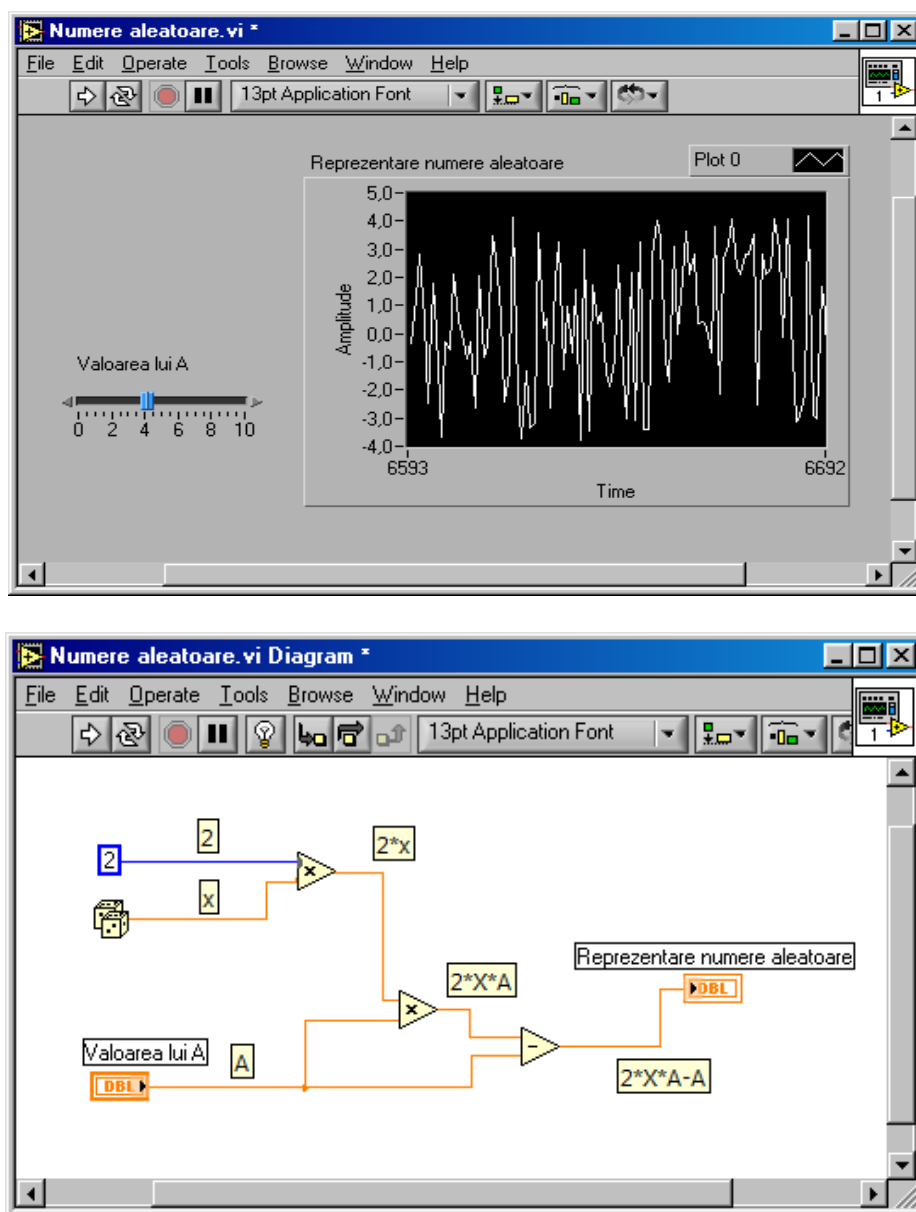


Figura 1.13 Panoul și diagrama programului pentru generarea numerelor aleatoare.

Indicații:

- o se deschide o nouă fereastră panou și se salvează cu numele *NumereAleatoare.vi*;

- o Se deschide panoul **Controls** (din *Windows / Show Controls Palette*), iar din **Numeric** se va face click pe **Simple Horizontal Slide** și se poziționează pe ecran (se va eticheta: *Valoarea lui A*);
- o În panoul **Controls** se selectează **Graph** și apoi **Waveform Chart**, se poziționează pe panou și se denumește: *Vizualizare numere aleatoare*;
- o Se activează fereastra **Diagram**, iar din meniul *Functions / Numeric* se aduc pe ecran următoarele icon-uri: **Random Number**, **Numeric Constant**, **Multiply** și **Subtract**;
- o Se cablează schema bloc din fereastra **Diagram** cu **Conect Wire** (din **Tools Palette**) pentru a realiza relația $y = 2Ax - A$. O conexiune corectă este indicată printr-un fir continuu. Dacă o conexiune devine punctată înseamnă că a apărut o eroare la conectare, iar eroarea respectivă este semnalizată într-o fereastră numită **Error List**;
- o După realizarea corectă a schemei, din fereastra panou se rulează aplicația, cu ajutorul butonului de *rulare* .

Observație: programul calculează un singur număr aleator la o rulare. Pentru a vedea mai multe numere pe grafic se poate apăsa butonul *rulare continuă* . O altă soluție este să se folosească o buclă de repetiție în program.

2. Să se modifice programul de la punctul anterior astfel încât la o singură rulare să se genereze 150 de numere aleatoare la un interval de 100ms unul de altul. Programul se va salva cu un alt nume (figura 1.14).

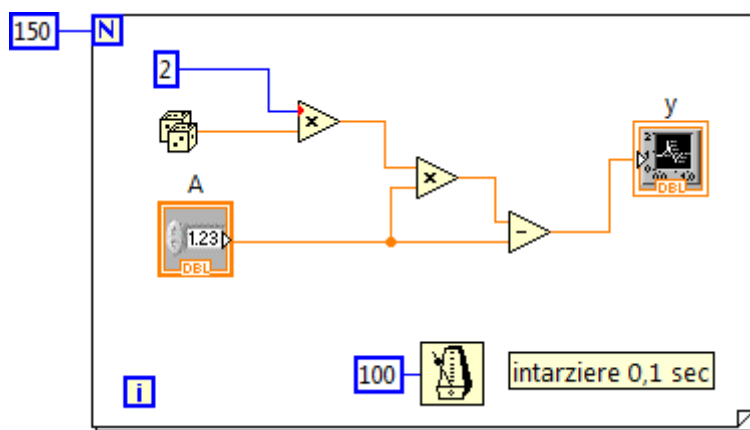


Figura 1.14 Diagrama modificată a programului pentru generarea numerelor aleatoare

3. Să se salveze VI-ul de la punctul 2 ca subVI cu numele *Numere aleatoare (-A, A)*. SubVI-ul va avea ca intrări: Numărul de numere generate (N), Valoarea lui A și perioada de generare (delay). Se va salva într-o librărie nouă. Indicatorul de reprezentare grafică a numerelor aleatoare se va scoate în exteriorul buclei *For*. Rulați aplicația și observați diferența față de punctul 2.

Justificați și explicați funcționarea aplicației în ambele cazuri și desenați organigrama logică de funcționare în cele două situații.

Indicație: pentru a crea un subVI, trebuie construite pictograma și conectorii pentru acel subVI. Conectorul reprezintă interfața programabilă către un VI. Dacă se utilizează controale sau indicatoare în panoul frontal pentru a schimba date cu subVI-urile, atunci aceste controale și indicatoare necesită terminale în zona conectorului.

Pentru realizarea *pictogramei* trebuie făcuți următorii pași:

- click dreapta pe pictograma (*icon*) din colțul din partea dreaptă sus a ferestrei panoului sau diagramei și selectarea opțiunii **Edit Icon**;
- alegerea numărului de culori: 16 sau 256 culori sau alb/negru și desenarea pictogramei folosind instrumentele din partea stângă.

Pentru realizarea *conectorilor* trebuie făcuți următorii pași:

- selectarea **Show Connector** din meniul cu comenzi rapide asociat zonei pictogramei;
- click dreapta pe conector și selectarea **Patterns** pentru a alege un alt mod de dispunere a terminalelor pentru VI;
- *Asocierea terminalelor la controale și indicatoare* – se face click pe un terminal al conectorului (terminalul se înnegrește) și apoi se face click pe controlul sau indicatorul panoului frontal la care se dorește asocierea terminalului selectat. În jurul controlului selectat va apare un marcaj;
- Se repetă pasul anterior pentru fiecare control sau indicator care trebuie conectat.

Lucrul cu vectori și cu fișiere

4. Să se scrie un program care să genereze un vector real cu 50 de componente aleatoare cu valori între -20 și 20. Pentru generarea valorilor aleatoare va fi folosit subVI-ul creat la punctul 2. Componentele vectorului se vor afișa pe un grafic și vor fi salvate pe coloană într-un fișier text pe harddisk

5. Să se realizeze un program care să genereze doi vectori coloană cu câte 10 componente reale, introduse de la tastatură din panoul frontal. Se vor calcula și afișa maximul și minimul din fiecare vector, și se vor ordona crescător cei doi vectori. Componentele celor doi vectori sortați vor fi apoi afișate pe același grafic și se vor salva într-un fișier tip text pe harddisk. Tot în același fișier se va salva și forma inițială a celor doi vectori.

6. Să se creeze un VI care să genereze două semnale: sinusoidal și cosinusoidal, fiecare semnal fiind format din câte 100 de eșantioane într-o perioadă. Amplitudinea semnalului sinusoidal va fi de 0.5, iar a celui cosinusoidal de 1. Semnalele se vor reprezenta grafic, sub formă tabelară, iar valorile eșantioanelor se vor scrie într-un fișier text pe două coloane. Panoul și diagrama programului sunt reprezentate în figura 1.15.

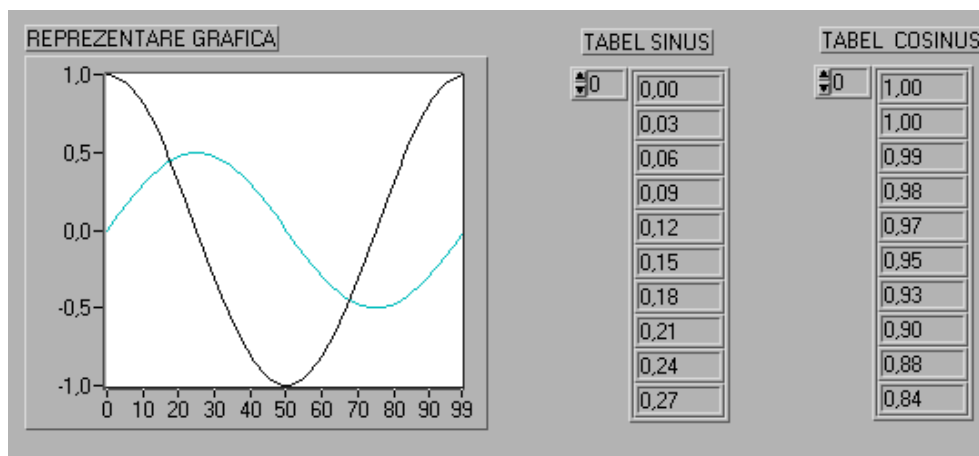


Figura 1.15 Panoul și diagrama programului pentru salvarea în fișier a două semnale

Indicație: se va forma o matrice bidimensională care să conțină valorile eșantioanelor celor două funcții iar apoi se va salva întreaga matrice în fișier folosind VI-ul *Write to Spreadsheet File.vi* din biblioteca *File I/O*.

7. Construiți un VI care să simuleze măsurarea continuă a temperaturii la intervale de o secundă și să o afișeze într-o diagramă cu actualizare prin redesenare. Dacă temperatura crește sau scade sub anumite valori prestabilite, VI-ul aprinde niște LED-uri de avertizare pe panoul frontal. Pe grafic vor fi afișate atât evoluția temperaturii cât și limitele permise de variație a acesteia. Limitele se vor stabili din panoul frontal și vor putea fi modificate în timpul rulării programului, cu ajutorul unui control *Knob*. Programul va rula continuu, până la apăsarea unui buton de pe panoul frontal, iar temperatura curentă se va afișa numeric și sub formă de termometru. Simularea măsurării temperaturii se va face cu ajutorul unui buton *Horizontal Pointer Slide* situat în panoul frontal. Denumiți VI-ul **Simulare monitorizare temperatură.vi**.

Indicație:

- pentru măsurarea repetată până la apăsarea unui buton se va folosi o buclă *While*;
- pentru afișarea datelor pe grafic se va folosi gruparea datelor de tip cluster
- o variantă de realizare a aplicației este următoarea: (figura 1.16)

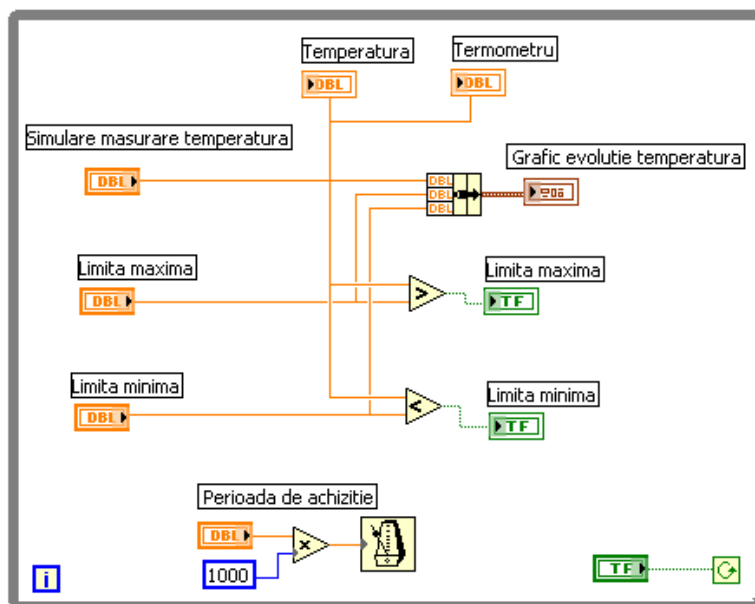
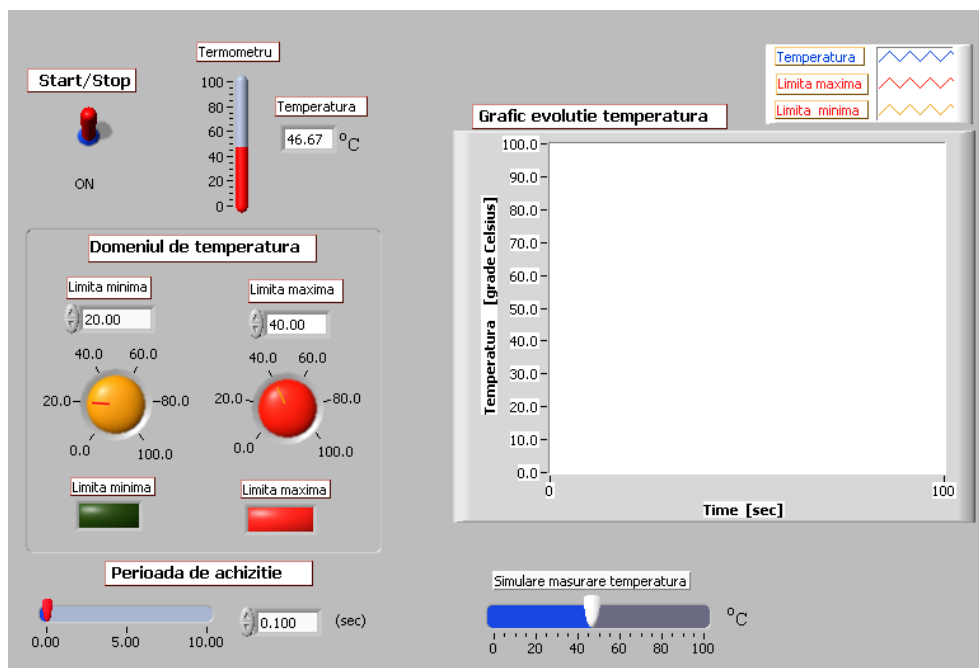


Figura 1.16 Panoul și diagrama VI-ului pentru simularea monitorizării temperaturii

8. Modificați VI-ul creat la exercițiul 7 pentru putea să salveze într-un fișier text pe harddisk, în ordine, următorii parametri: data și ora efectuării măsurătorii, temperatura măsurată, limita minimă impusă, limita maximă impusa, dacă se încadrează sau nu în domeniul impus.

Indicație:

- pentru aflarea datei și orei la care s-a preluat eșantionul se va folosi funcția *Get Date/Time String* din librăria *Time & Dialog*;
- se va face conversia datelor din cluster în string;
- se va compune șirul cu datele care trebuie salvate pe o linie în fișier, în ordinea specificată folosind funcții de operare cu șiruri;
- se va scrie în fișier folosind funcții specializate sau VI-ul *Write Characters To File.vi*.

Studiarea unor programe propuse cu rol de exemplu

9. Se vor studia următoarele programe demonstrative în LabVIEW:

- *Control Mixer Process.vi*
- *Temperature System Demo.vi*
- *generarea și procesarea semnalelor*
- *filtru de mediere.*

Pentru fiecare program se va analiza funcționarea și modul de realizare (diagrama). Se va studia structura ierarhică a acestora, deschizând fiecare subVI din structura acestora și se va analiza modul în care a fost realizată funcția respectivă. Ierarhia VI-ului se poate vizualiza accesând meniul *Browse → Show VI Hierarchy*

Temă

1. Să se realizeze un program LabVIEW care să rezolve ecuațiile de ordinul 1 și 2. Coeficienții se introduc de la tastatură, iar programul va calcula rădăcinile ecuației. Se va trasa graficul funcției de gradul 2 .
2. Să se realizeze un program care să traseze caracteristicile de încărcare și descărcare ale unui condensator dintr-un circuit RC. VI-ul va reprezenta grafic tensiunea și curentul prin condensator în funcție de timp, va calcula și reprezenta constanta de timp a circuitului, precum și curentul prin condensator în funcție de tensiunea de la bornele acestuia. Pentru simulare, la bornele circuitului se va aplica o rampă de tensiune cu valoarea E reglabilă. Momentul aplicării tensiunii (închiderea comutatorului care realizează cuplarea circuitului RC la sursa de tensiune E) va putea fi stabilit de utilizator, prin apăsarea unui buton în panoul frontal. Tot din panou se vor putea introduce valorile rezistorului și al condensatorului.

Observație: Referatul va fi scris pe calculator și va conține aplicațiile propuse la punctele *desfășurarea lucrării și temă*. Pentru fiecare aplicație se va prezenta diagrama bloc și panoul frontal, captat în timpul rulării, cu reprezentarea grafică a semnalelor, precum și o scurtă descriere a modului de realizare și observații.

De asemenea, pentru fiecare program se va prezenta organigrama corespunzătoare.